

회원 관리 프로그램 만들기

이 문서는 자바로 간단한 **회원 관리 프로그램**을 만드는 과정을 설명한다.

이 프로그램은 회원의 **아이디**, **이름**, **나이** 를 저장하고, 등록된 회원 목록을 출력하는 예제다. 처음에는 변수만 사용해서 단순하게 시작한 뒤, 점점 **Member** 클래스, 배열, **count**, **MemberManager** 클래스로 발전시킨다.

최종적으로는 아래와 같은 구조를 목표로 한다.

Main

→ 프로그램 실행 담당

Member

→ 회원 한 명의 정보 담당

MemberManager

→ 회원 여러 명 등록 및 출력 담당

이 예제의 핵심은 완성 코드를 외우는 것이 아니라, 개발자가 왜 클래스를 나누고 메서드를 만드는지 흐름을 이해하는 것이다.

개발자라면 생각하는 순서

처음부터 **MemberManager**, **private**, **count**, **printMembers()** 를 한 번에 떠올리는 것이 아니다. 보통은 아래처럼 생각이 변한다.

1단계: 회원 한 명만 만들어서 출력

처음 목표는 단순하다.

회원 한 명의 정보를 출력해보자.

```
package ex06;

public class Main {
    public static void main(String[] args) {
        String id = "user1";
        String name = "홍길동";
        int age = 20;

        System.out.println("아이디: " + id + ", 이름: " + name + ", 나이: " + age);
    }
}
```

```
}  
}
```

이 단계에서는 클래스 설계가 중요한 것이 아니다.

핵심은 이것이다.

회원 정보에는 `id`, `name`, `age` 가 필요하구나.

2단계: 회원이 여러 명이면 불편함을 느끼기

회원이 2명이 되면 코드가 반복된다.

```
package ex06;  
  
public class Main {  
    public static void main(String[] args) {  
        String id1 = "user1";  
        String name1 = "홍길동";  
        int age1 = 20;  
  
        String id2 = "user2";  
        String name2 = "김자바";  
        int age2 = 25;  
  
        System.out.println("아이디: " + id1 + ", 이름: " + name1 + ", 나이: " + age1);  
        System.out.println("아이디: " + id2 + ", 이름: " + name2 + ", 나이: " + age2);  
    }  
}
```

여기서 개발자는 이런 생각을 한다.

`id`, `name`, `age` 는 따로 노는 값이 아니라 회원 한 명의 정보로 묶여야겠네.

그래서 `Member` 클래스를 만들게 된다.

3단계: `Member` 클래스로 회원 한 명 묶기

처음에는 `private` 까지 안 가도 된다.

```
package ex06;
```

```
public class Member {
    String id;
    String name;
    int age;
}
```

Main에서는 이렇게 사용한다.

```
package ex06;

public class Main {
    public static void main(String[] args) {
        Member member1 = new Member();

        member1.id = "user1";
        member1.name = "홍길동";
        member1.age = 20;

        System.out.println("아이디: " + member1.id + ", 이름: " + member1.name + ", 나이: " +
            member1.age);
    }
}
```

이 단계의 핵심은 이것이다.

관련 있는 데이터를 하나의 객체로 묶었다.

즉, 이제 회원 한 명은 변수 3개가 아니라 Member 객체 하나가 된다.

4단계: 출력 기능을 Member 안으로 넣기

지금은 출력할 때마다 이 코드가 길어진다.

```
System.out.println("아이디: " + member1.id + ", 이름: " + member1.name + ", 나이: " + member1.age);
```

그래서 개발자는 이런 생각을 한다.

회원 정보를 출력하는 기능은 회원 객체가 직접 갖고 있으면 좋겠다.

그래서 Member에 printInfo()를 넣는다.

```
package ex06;
```

```
public class Member {
    String id;
    String name;
    int age;

    public void printInfo() {
        System.out.println("아이디: " + id + ", 이름: " + name + ", 나이: " + age);
    }
}
```

Main 은 이렇게 바뀐다.

```
package ex06;

public class Main {
    public static void main(String[] args) {
        Member member1 = new Member();

        member1.id = "user1";
        member1.name = "홍길동";
        member1.age = 20;

        member1.printInfo();
    }
}
```

이제 출력 책임이 바뀐다.

```
member1.printInfo();
```

의 의미는 다음과 같다.

회원 객체야, 네 정보를 출력해라.

5단계: 생성자로 값을 한 번에 넣기

지금은 객체 만들고 나서 값을 따로 넣어야 한다.

```
Member member1 = new Member();

member1.id = "user1";
```

```
member1.name = "홍길동";  
member1.age = 20;
```

개발자는 이걸 보고 이렇게 생각한다.

회원 객체는 만들 때부터 id, name, age가 있어야 자연스럽다.

그래서 생성자를 만든다.

```
package ex06;  
  
public class Member {  
    String id;  
    String name;  
    int age;  
  
    public Member(String id, String name, int age) {  
        this.id = id;  
        this.name = name;  
        this.age = age;  
    }  
  
    public void printInfo() {  
        System.out.println("아이디: " + id + ", 이름: " + name + ", 나이: " + age);  
    }  
}
```

이제 `Main` 은 짧아진다.

```
package ex06;  
  
public class Main {  
    public static void main(String[] args) {  
        Member member1 = new Member("user1", "홍길동", 20);  
  
        member1.printInfo();  
    }  
}
```

이 단계에서 생성자는 이렇게 설명하면 된다.

생성자는 객체를 만들 때 필요한 값을 넣어주는 통로다.

6단계: 회원 여러 명을 배열에 담기

회원이 여러 명이면 `Member` 변수를 계속 만들 수도 있다.

```
Member member1 = new Member("user1", "홍길동", 20);
Member member2 = new Member("user2", "김자바", 25);
Member member3 = new Member("user3", "이순신", 30);
```

근데 회원 수가 많아지면 관리가 힘들다.

개발자는 이렇게 생각한다.

같은 종류의 객체가 여러 개니까 배열에 담자.

```
package ex06;

public class Main {
    public static void main(String[] args) {
        Member[] members = new Member[10];

        members[0] = new Member("user1", "홍길동", 20);
        members[1] = new Member("user2", "김자바", 25);

        members[0].printInfo();
        members[1].printInfo();
    }
}
```

여기서 중요한 것은 이것이다.

```
Member[] members = new Member[10];
```

이건 `Member` 객체 10개를 만든 것이 아니다.

`Member` 객체를 담을 수 있는 칸 10개를 만든 것이다.

실제 객체는 여기서 만들어진다.

```
members[0] = new Member("user1", "홍길동", 20);
members[1] = new Member("user2", "김자바", 25);
```

7단계: 출력도 반복문으로 바꾸기

지금은 출력도 직접 번호를 써야 한다.

```
members[0].printInfo();  
members[1].printInfo();
```

회원이 10명이면 너무 귀찮다.

그래서 반복문으로 바꾼다.

```
package ex06;  
  
public class Main {  
    public static void main(String[] args) {  
        Member[] members = new Member[10];  
  
        members[0] = new Member("user1", "홍길동", 20);  
        members[1] = new Member("user2", "김자바", 25);  
  
        for (int i = 0; i < 2; i++) {  
            members[i].printInfo();  
        }  
    }  
}
```

그런데 여기서 문제가 하나 생긴다.

```
for (int i = 0; i < 2; i++)
```

왜 2일까?

지금은 회원이 2명이라서 2지만, 나중에 회원 수가 바뀌면 이 숫자도 바뀌어야 한다.

개발자는 이렇게 생각한다.

실제 등록된 회원 수를 따로 기억해야겠네.

그래서 `count`가 나온다.

8단계: `count`로 실제 회원 수 관리하기

배열 크기는 10이지만, 실제 등록된 회원은 2명이다.

그래서 `count`를 둔다.

```
package ex06;

public class Main {
    public static void main(String[] args) {
        Member[] members = new Member[10];
        int count = 0;

        members[count] = new Member("user1", "홍길동", 20);
        count++;

        members[count] = new Member("user2", "김자바", 25);
        count++;

        for (int i = 0; i < count; i++) {
            members[i].printInfo();
        }
    }
}
```

이 코드가 중요하다.

```
members[count] = new Member("user1", "홍길동", 20);
count++;
```

처음에는 `count`가 0이다.

그래서 0번 칸에 회원을 넣는다.

```
members[0] = new Member("user1", "홍길동", 20);
```

그리고 `count++` 해서 실제 등록된 회원 수를 1로 만든다.

두 번째 회원은 1번 칸에 들어간다.

```
members[1] = new Member("user2", "김자바", 25);
```

그리고 `count++` 해서 실제 등록된 회원 수는 2가 된다.

그래서 출력 반복문은 이렇게 된다.

```
for (int i = 0; i < count; i++)
```

뜻은 다음과 같다.

0번부터 실제 등록된 회원 수 전까지 출력해라.

9단계: Main 이 너무 복잡하니 MemberManager 만들기

지금 Main 이 너무 많은 일을 하고 있다.

Main 이 하는 일:

```
회원 배열 만들기  
count 관리하기  
회원 추가하기  
회원 목록 출력하기
```

개발자는 이렇게 생각한다.

회원을 관리하는 코드는 따로 빼자.

그래서 MemberManager 를 만든다.

처음 버전은 이렇게 시작할 수 있다.

```
package ex06;  
  
public class MemberManager {  
    Member[] members = new Member[10];  
    int count = 0;  
}
```

이 클래스의 의미는 다음과 같다.

회원 여러 명을 관리하는 클래스

10단계: 회원 추가 기능을 addMember() 로 빼기

아까 Main 에 있던 이 코드:

```
members[count] = new Member("user1", "홍길동", 20);  
count++;
```

이걸 매번 쓰기 귀찮다.

그래서 `MemberManager` 안에 넣는다.

```
package ex06;  
  
public class MemberManager {  
    Member[] members = new Member[10];  
    int count = 0;  
  
    public void addMember(Member member) {  
        members[count] = member;  
        count++;  
    }  
}
```

이제 `Main` 은 이렇게 된다.

```
package ex06;  
  
public class Main {  
    public static void main(String[] args) {  
        MemberManager manager = new MemberManager();  
  
        manager.addMember(new Member("user1", "홍길동", 20));  
        manager.addMember(new Member("user2", "김자바", 25));  
    }  
}
```

이제 의미가 좋아진다.

```
manager.addMember(new Member("user1", "홍길동", 20));
```

뜻은 다음과 같다.

회원관리자야, 이 회원을 등록해라.

11단계: 회원 목록 출력 기능을 printMembers()로 빼기

아까 Main에 있던 출력 코드:

```
for (int i = 0; i < count; i++) {  
    members[i].printInfo();  
}
```

이건 회원관리자가 해야 할 일이다.

왜냐하면 members 배열과 count를 알고 있는 쪽이 MemberManager니까.

그래서 이 메서드를 넣는다.

```
public void printMembers(){  
    System.out.println("회원 목록");  
  
    for(int i=0; i<count; i++){  
        members[i].printInfo();  
    }  
}
```

전체 MemberManager는 이렇게 된다.

```
package ex06;  
  
public class MemberManager {  
    Member[] members = new Member[10];  
    int count = 0;  
  
    public void addMember(Member member) {  
        members[count] = member;  
        count++;  
        System.out.println("회원이 등록되었습니다.");  
    }  
  
    public void printMembers(){  
        System.out.println("회원 목록");  
  
        for(int i=0; i<count; i++){  
            members[i].printInfo();  
        }  
    }  
}
```

```
}  
}
```

Main 은 이렇게 깔끔해진다.

```
package ex06;  
  
public class Main {  
    public static void main(String[] args) {  
        MemberManager manager = new MemberManager();  
  
        manager.addMember(new Member("user1", "홍길동", 20));  
        manager.addMember(new Member("user2", "김자바", 25));  
  
        manager.printMembers();  
    }  
}
```

이제 구조가 명확해진다.

Main
→ 프로그램 실행 담당

Member
→ 회원 한 명의 정보 담당

MemberManager
→ 회원 여러 명 관리 담당

12단계: 배열이 꽉 찼을 때 방어 코드 추가

지금 `addMember()` 는 문제가 있다.

```
public void addMember(Member member) {  
    members[count] = member;  
    count++;  
}
```

만약 11번째 회원을 넣으면?

배열은 10칸뿐이라서 에러가 난다.

그래서 개발자는 이렇게 생각한다.

회원을 추가하기 전에 배열이 꽉 찼는지 검사해야겠다.

그래서 이렇게 바꾼다.

```
public void addMember(Member member) {  
    if (count >= members.length) {  
        System.out.println("더 이상 회원을 등록할 수 없습니다.");  
        return;  
    }  
  
    members[count] = member;  
    count++;  
    System.out.println("회원이 등록되었습니다.");  
}
```

이제 안전해진다.

13단계: 외부에서 배열과 count를 못 건드리게 private

여기까지 된 다음에 `private` 을 넣으면 자연스럽다.

현재는 외부에서 이런 짓이 가능하다.

```
manager.count = 100;
```

또는

```
manager.members[0] = null;
```

그러면 회원 관리가 망가진다.

그래서 개발자는 이렇게 생각한다.

`members` 와 `count` 는 `MemberManager` 내부에서만 관리해야겠다.

그래서 `private` 을 붙인다.

```
private Member[] members = new Member[10];  
private int count = 0;
```

완성형 `MemberManager` 는 이렇게 된다.

```
package ex06;

public class MemberManager {
    private Member[] members = new Member[10];
    private int count = 0;

    public void addMember(Member member) {
        if (count >= members.length) {
            System.out.println("더 이상 회원을 등록할 수 없습니다.");
            return;
        }

        members[count] = member;
        count++;
        System.out.println("회원이 등록되었습니다.");
    }

    public void printMembers(){
        System.out.println("회원 목록");

        for(int i=0; i<count; i++){
            members[i].printInfo();
        }
    }
}
```

최종 완성본

Main.java

```
package ex06;

public class Main {
    public static void main(String[] args) {
        MemberManager manager = new MemberManager();

        manager.addMember(new Member("user1", "홍길동", 20));
        manager.addMember(new Member("user2", "김자바", 25));

        manager.printMembers();
    }
}
```

```
}  
}
```

Member.java

초보 수업용이면 getter/setter 없이 이 정도로도 충분하다.

```
package ex06;  
  
public class Member {  
    private String id;  
    private String name;  
    private int age;  
  
    public Member(String id, String name, int age) {  
        this.id = id;  
        this.name = name;  
        this.age = age;  
    }  
  
    public void printInfo() {  
        System.out.println("아이디: " + id + ", 이름: " + name + ", 나이: " + age);  
    }  
}
```

MemberManager.java

```
package ex06;  
  
public class MemberManager {  
    private Member[] members = new Member[10];  
    private int count = 0;  
  
    public void addMember(Member member) {  
        if (count >= members.length) {  
            System.out.println("더 이상 회원을 등록할 수 없습니다.");  
            return;  
        }  
  
        members[count] = member;  
        count++;  
        System.out.println("회원이 등록되었습니다.");  
    }  
}
```

```
public void printMembers(){
    System.out.println("회원 목록");

    for(int i=0; i<count; i++){
        members[i].printInfo();
    }
}
}
```

수업에서 설명하는 흐름

이렇게 말하면 좋다.

처음에는 회원 정보를 변수로 따로 만들었습니다.
 그런데 `id`, `name`, `age` 는 회원 한 명의 정보라서 `Member` 클래스로 묶었습니다.
 회원이 여러 명이 되니까 `Member[]` 배열에 담았습니다.
 배열에 몇 명이 들어갔는지 알아야 해서 `count` 를 만들었습니다.
 회원 추가와 출력 코드가 길어져서 `MemberManager` 클래스로 분리했습니다.
 마지막으로 `members` 배열과 `count` 는 외부에서 건드리면 안 되니까 `private` 으로 막았습니다.

학생들에게 꼭 보여줄 핵심 변화

처음에는 이랬다.

```
members[count] = new Member("user1", "홍길동", 20);
count++;
```

이게 나중에는 이렇게 바뀐다.

```
manager.addMember(new Member("user1", "홍길동", 20));
```

처음에는 이랬다.

```
for (int i = 0; i < count; i++) {
    members[i].printInfo();
}
```

이게 나중에는 이렇게 바뀐다.


```
manager.printMembers();
```

즉, 개발자가 한 일은 어려운 것이 아니라 이것이다.

자주 쓰는 코드를 메서드로 묶고, 관련 있는 코드를 클래스로 분리한 것.

이게 객체지향 설계의 시작이라고 보면 된다.